

Advanced Data Structures COP5536, Fall 2023

Programming Project Report

Name: Phalguna Peravali

UFID: 3753-9361

Email: phalgunaperavali@ufl.edu

Under the Guidance of
Dr. Sartaj Sahni

Introduction:

The Gator Library project is a Python-based library management system designed to streamline library operations and enhance user satisfaction. It employs a modular architecture, utilizing distinct classes for books, patrons, and the main script, making it maintainable and scalable.

Red-Black Tree:

Red-Black Trees are a type of self-balancing binary search tree that maintains certain properties to ensure efficient search, insertion, and deletion operations. At the heart of the Gator Library project lies the Red-Black Tree, a sophisticated data structure that maintains the order of books, ensuring efficient searching and retrieval. It performs color flips during its operations to preserve its self-balancing properties, guaranteeing optimal performance even as the book collection grows.

MinHeap:

The Minheap serves as a VIP line for patrons, prioritizing their access to books. It prioritizes patrons based on their assigned priority values, ensuring that those with the highest urgency get served first. The Minheap's quick insertion, removal, and deletion capabilities further expedite the reservation process.

Book Class:

The Book Class encapsulates all the essential information about a book, including its title, author, ISBN, borrowing status, and reservation details. It maintains a record of who has borrowed the book and manages the list of patrons waiting to borrow it, ensuring that the book's whereabouts and availability are always known.

Patron Class:

The Patron Class represents the library's patrons, storing their unique IDs, contact information, and priority levels. It plays a crucial role in determining which patron gets priority when a book becomes available, ensuring that the system adheres to fairness and efficiency principles.

Main Script:

The Main Script serves as the central orchestrator of the system, reading commands from an input file and translating them into actionable tasks. It executes a wide range of operations, including inserting new books, searching for specific titles, processing

borrowing and returning requests and managing reservations. The Main Script generates detailed output, providing users with comprehensive information about the library's operations.

Project Flow:

The Gator Library project seamlessly integrates its components to create a cohesive and efficient system. The Main Script acts as the central hub, receiving commands and delegating tasks to the Red-Black Tree, MinHeap, Book Class, and Patron Class. These components work in harmony to maintain a well-organized library, ensuring that books are easily searchable, patrons are prioritized, and library activities are smoothly managed.

I have implemented the following functions and will explain the time complexities of the following functions.

1. MinHeap Class

Methods:

init (self):

- Responsibility: Initializes an empty min-heap.
- Complexity: $O(1)$

str (self):

- Responsibility: Returns a string representation of the min-heap.
- Complexity: $O(n)$, where n is the number of elements in the heap.

is_empty(self):

- Responsibility: Checks if the min-heap is empty.
- Complexity: $O(1)$

get_all_elements(self):

- Responsibility: Returns all elements in the min-heap.
- Complexity: $O(n)$, where n is the number of elements in the heap.

insert(self, node):

- Responsibility: Inserts a node into the min-heap and maintains the heap property.
- Complexity: $O(\log n)$, where n is the number of elements in the heap.

get_min(self):

- Responsibility: Returns the minimum element in the min-heap.
- Complexity: $O(1)$

remove_min(self):

- Responsibility: Removes and returns the minimum element from the min-heap.
- Complexity: $O(\log n)$, where n is the number of elements in the heap.

delete(self, node):

- Responsibility: Deletes a specific node from the min-heap.
- Complexity: $O(n)$, where n is the number of elements in the heap (due to searching for the node).

2. Patron Class

Methods:

init (self, patronId, patronPriority):

- Responsibility: Initializes a patron with a unique ID and priority.
- Complexity: $O(1)$

compare (self, other):

- Responsibility: Compares the priority of two patrons.
- Complexity: $O(1)$

str (self):

- Responsibility: Returns a string representation of the patron.
- Complexity: $O(1)$

get (self, other):

- Responsibility: Checks if the priority of the current patron is greater than another patron.
- Complexity: $O(1)$

3. Book Class

Methods:

init(self, bookId, bookName, authorName, availabilityStatus):

- Responsibility: Initializes a book with specific attributes.
- Complexity: $O(1)$

insert_into_reservation(self, patronId, patronPriority):

- Responsibility: Inserts a patron into the book's reservation heap.
- Complexity: $O(\log n)$, where n is the number of patrons in the reservation heap.

borrowBook(self, patronId, patronPriority):

- Responsibility: Handles the borrowing of a book by a patron.
- Complexity: $O(\log n)$, where n is the number of patrons in the reservation heap.

returnBook(self, patronId):

- Responsibility: Handles the return of a book by a patron, updating availability and reservations.
- Complexity: $O(\log n)$, where n is the number of patrons in the reservation heap.

str(self):

- Responsibility: Returns a string representation of the book.
- Complexity: $O(1)$

4. RedBlackTree Class

Methods:

init (self):

- Responsibility: Initializes an empty Red-Black Tree.

- Complexity: $O(1)$

search_tree_helper(self, node, key):

- Responsibility: Searches the tree for a specific book ID.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

delete_fix(self, x):

- Responsibility: Fixes the tree after a node deletion to maintain Red-Black Tree properties.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

rb_transplant(self, u, v):

- Responsibility: Replaces one subtree with another during deletion.
- Complexity: $O(1)$

delete_node_helper(self, node, key):

- Responsibility: Deletes a node with a specific book ID from the tree.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

fix_insert(self, k):

- Responsibility: Fixes the tree after a node insertion to maintain Red-Black Tree properties.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

minimum (self, node):

- Responsibility: Finds the node with the minimum key in the subtree rooted at the given node.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

left_rotate(self, x):

- Responsibility: Performs a left rotation operation on the tree.
- Complexity: $O(1)$

right_rotate(self, x):

- Responsibility: Performs a right rotation operation on the tree.
- Complexity: $O(1)$

closest_node(self, node, bookId, min_diff, min_diff_book):

- Responsibility: Finds the closest node to a given book ID.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

findClosestBook(self, targetId):

- Responsibility: Finds the closest book(s) to a target book ID.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

insertBook(self, bookId, bookName, authorName, availabilityStatus):

- Responsibility: Inserts a new book into the tree.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

deleteBook(self, bookId):

- Responsibility: Deletes a book from the tree and cancels reservations.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

searchBook(self, bookId):

- Responsibility: Searches for a book in the tree based on its ID.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

printBook(self, bookId):

- Responsibility: Prints information about a specific book.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

printBooks(self, bookId1, bookId2):

- Responsibility: Prints information about all books within a specified range.
- Complexity: $O(k + \log n)$, where k is the number of books in the specified range and n is the number of nodes in the tree.

borrowBook(self, patronId, bookId, patronPriority):

- Responsibility: Borrows a book for a patron.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

returnBook(self, patronId, bookId):

- Responsibility: Returns a borrowed book and handles reservations.
- Complexity: $O(\log n)$, where n is the number of nodes in the tree.

colorFlipCount(self):

- Responsibility: Returns the count of color flips in the Red-Black Tree.
- Complexity: $O(1)$

quit(self):

- Responsibility: Prints a termination message.
- Complexity: $O(1)$

Main Program

Responsibility: Reads input from a file, performs library operations, and writes output to another file.

Flow:

Reads each line from the input file.

Parses the input and calls corresponding methods of the RedBlackTree class.

Writes the output to the output file.

Accepts commands through a text file for operations like book insertion, printing, borrowing, returning, finding the closest book, and deletion.

Detailed output is generated, providing insights into performed operations, book details, and system status.

Workflow:

- Initialization: Initialize data structures and process command-line arguments.
- Command Processing: Execute operations based on input commands.
- Library Operations: Insert, print, borrow, return, manage reservations, and count color flips.
- Output Generation: Write detailed output to a file.
- Termination: Program concludes upon encountering the "Quit" command.

I have listed the time complexity of each function that we were asked to implement. I have also explained the reasoning behind the time complexities.